

CGI Capacity Analyzer for Director Workload and RFP Assignment

Freya Kan, Jai, Junxian Lin, Qian Yang, Yixiao Jing

Master of Data Science in Computational Linguistics

University of British Columbia

Abstract

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. We develop a Capacity Analyzer that combines historical CRM opportunity data with proposal and RFP text to support director capacity estimation and RFP assignment. The system is designed as a transparent decision-support tool rather than a replacement for business judgment. Week 1 established the reproducible opportunity data foundation and module skeletons. Week 2 extended this foundation into a first integrated prototype by refining the cleaned opportunity dataset, validating scoring fields, implementing director-level capacity scoring, connecting capacity outputs to the Streamlit dashboard, and advancing the RFP preprocessing pipeline. Week 3 refined the capacity dashboard around a relative-load-first interpretation and shifted the main focus toward RFP assignment, including fallback retrieval, assignment schema stabilization, heuristic recommendation logic, and Streamlit RFP page integration. Week 4 expanded the prototype into a final stakeholder-facing version with real capacity-aware RFP recommendations, Azure OpenAI chat enrichment for RFP summaries, effort estimates, and director match reasons, Azure OpenAI embedding and Chroma retrieval integration with local fallback preserved, RFP upload support alongside paste input, clearer fallback and prototype-status labeling, and stronger empty-state, error-state, and response-quality checks. Week 5 focused on final app QA and refinement rather than major new feature development: the team validated Azure/Chroma retrieval and fallback behavior, reviewed whether retrieved examples supported displayed RFP outputs, refined assignment caveats and risk wording, improved stakeholder-facing labels and LLM/retrieval status captions, and consolidated final scope, test, and caveat tracking. Week 6 shifted the project into client handoff support and data replacement readi-

ness. The team updated the opportunity preprocessing workflow so CGI can run the cleaner with local spreadsheet paths through command-line inputs while preserving the original demo-data workflow; preserved CGI's new machine-readable RFP alias field as `rfp_alias` and `has_rfp_alias`; added and reviewed regression coverage for configurable inputs, alias preservation, and linked metadata propagation; validated that the app can continue loading processed CRM-derived capacity data after regeneration; and refined documentation, troubleshooting guidance, and client-facing wording for handoff. The resulting prototype is better prepared for CGI local testing and review, but it remains a transparent stakeholder-facing decision-support system rather than a production-ready or business-validated assignment system.

1 Introduction

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. Today, capacity may be tracked manually or inconsistently, making it difficult to see which directors are available, near their historical workload norm, overloaded, or well matched to a new opportunity.

This project builds a CGI Capacity Analyzer to support two related business decisions. First, the Director Capacity Dashboard summarizes each director's current workload, historical baseline, relative load, capacity label, and workload drivers. In the Week 6 handoff-ready prototype, `relative_load` remains the primary business-facing metric because it compares a director's current workload against that same director's historical norm. Second, the RFP Assignment Tool estimates RFP effort, retrieves semantically similar historical proposal or response examples through the preferred Azure OpenAI embedding and Chroma retrieval path when available, and recommends suitable directors based on available capacity signals,

retrieved supporting examples, service-domain overlap, and risk. The local fallback retrieval path remains available when Azure or Chroma is unavailable, skipped, or unsuitable for the current request, and the RFP page reports the active retrieval mode so stakeholders can distinguish Azure/Chroma-backed retrieval from local fallback retrieval.

The system is not intended to replace stakeholder judgment. Instead, it provides a transparent and reproducible data-based starting point that CGI stakeholders can review, validate, and adjust. This design choice is important because the project does not currently have manually labeled ground truth for director capacity, RFP effort, or correct director assignment. The Week 6 prototype should therefore be interpreted as decision-support guidance: capacity-aware when real or pre-built capacity data is available, LLM-enriched when Azure chat configuration is active, and still supported by deterministic heuristic or fallback behavior when LLM enrichment or retrieval components are unavailable. Week 6 also improved handoff readiness by supporting command-line replacement of local opportunity spreadsheets, preserving CGI's machine-readable RFP alias field, and allowing linked CRM opportunity context to be surfaced when reliable alias-based linkage is available. However, retrieved and linked examples should still be treated as semantic evidence and CRM context related to the submitted RFP text, not as proof of prior director involvement, confirmed director experience, proposal authorship, CRM opportunity ownership, or final assignment suitability. Recommendations remain stakeholder-facing prototype outputs that require CGI review and validation before any staffing decision.

2 Data

2.1 Data Sources

The project uses two anonymized Excel CRM opportunity exports and one JSON file containing historical proposal and response text. The two opportunity exports are referred to as opps1 and opps2. The proposal data contains historical proposal and response document pairs. In Week 6, CGI provided an updated version of the opps1 spreadsheet with the same 8,435 data rows and one additional meaningful RFP alias column, observed as `Json S-Num`. This increased the meaningful column count from 20 to 21 for the updated file. The workbook may

contain additional blank or formatted columns, but these are not treated as data fields.

All CGI-provided files are treated as sensitive local data. They are not committed to GitHub. Local credentials and API configuration are also excluded from version control. Week 6 further reinforced this handoff design by supporting local spreadsheet replacement through command-line preprocessing inputs, so CGI can place client-provided spreadsheets under local data paths, regenerate processed outputs, and test the app without committing raw spreadsheets or generated processed data to the repository.

2.2 Opportunity Data

Initial EDA indicates that opps1 and opps2 are different views of largely overlapping CRM opportunity data. We use opps2 as the primary base table because it contains the richer CRM schema and has one unique `opportunity_id` per row in the current export. We use opps1 as a secondary source: selected fields from opps1 are collapsed to one row per `opportunity_id` and merged into opps2, while opps1-exclusive opportunities are appended to the final `opportunity_df`. Fields that appear under different names across the two exports, especially revenue-related fields, require normalization and validation before downstream scoring and are handled through the Week 2 validation and fallback process.

This distinction is important because the downstream capacity model is expected to operate at the opportunity level. During the Week 1 merge implementation, we found that opps1 contains duplicate `opportunity_id` values. These duplicates likely reflect service-level or product-level detail rather than distinct opportunities. A direct merge from opps1 into opps2 would therefore multiply some opportunity rows and risk double-counting workload in later capacity scoring.

To preserve the intended data grain, the merge pipeline collapses opps1 supplemental fields to one row per `opportunity_id` before joining them to opps2. Duplicate and detail rows from opps1 are preserved in local audit outputs rather than being used to expand the final `opportunity_df`. In Week 6, this same preprocessing design was extended for client data replacement: the opportunity cleaner now supports command-line paths for local opps1 and opps2 spreadsheets, a configurable sheet name, and a configurable processed-output directory, while preserving the original no-argument

Data source	Approx. size	Main contents	Role in project
opps1 CRM export	8,435 rows 20 original columns; 21 meaningful columns in Week 6 update	Supplemental product, service-solution, IP, service-solution revenue, opportunity-level fallback revenue, delivery territory fields, and, in the Week 6 updated file, a machine-readable RFP alias field observed as Json S-Num.	Secondary supplemental source. Selected fields are collapsed to one row per opportunity_id before merging into opps2. In Week 6, the alias field is preserved as rfp_alias / has_rfp_alias when present to support RFP-to-CRM linkage review.
opps2 CRM export	8,695 rows 33 columns	Richer CRM opportunity schema with owner, status, sales stage, probability, revenue, date, duration, and proposal-related fields.	Primary base table for opportunity_df. It defines the opportunity-level grain for capacity scoring and dashboard inputs.
Proposal / response JSON	19 document pairs	Historical proposal and response text.	Used for RFP text preprocessing, historical corpus chunking, retrieved supporting examples, assignment-context generation, Azure/Chroma backend retrieval validation, fallback-safe local retrieval, and alias-based linkage review when a matching rfp_alias is available.

Table 1: Overview of the main project data sources used through Week 6. The files are stored locally and are not committed to GitHub because they contain CGI-provided data.

demo workflow.

The Week 6 updated opps1 spreadsheet also introduced a new machine-readable RFP alias column, observed as Json S-Num. After column normalization, this field is treated as json_s_num and preserved in the cleaned opportunity output as rfp_alias, with has_rfp_alias indicating whether a row contains an alias. This preservation step is important because the opps1 merge uses a selected supplemental-field whitelist: without explicitly retaining the alias field, RFP alias values on records matched into opps2 could be dropped during preprocessing. The alias is used only as a machine-readable linkage field for review and downstream context; Excel cell highlighting, including orange-highlighted rows, is treated as a manual visual annotation and is not used as pipeline logic.

2.3 Proposal and RFP Text Data

The proposal JSON contains 19 historical proposal and response document pairs. These documents are text-heavy and many are too long to embed as single inputs. Therefore, the RFP pipeline prepares proposal text through text extraction, chunking, metadata construction, and retrieval-oriented processing. The historical or sample corpus is chunked

before retrieval so that retrieved evidence can be surfaced as smaller supporting examples rather than as full proposal documents.

By Week 5, the RFP pipeline loads and chunks the local proposals_responses.json corpus when available, with a built-in sample corpus used only as fallback. Submitted RFP text, whether pasted or extracted from TXT, DOCX, or text-based PDF upload, is used as query text against the chunked historical or sample corpus. The preferred retrieval path uses Azure OpenAI embeddings and Chroma when configured and suitable for the request, while deterministic local retrieval over the proposal corpus remains available when Azure / Chroma is unavailable, skipped, or fails. The pipeline then generates a dashboard-facing assignment_context for Streamlit display.

The proposal text supports the RFP Assignment Tool by enabling retrieval of semantically related historical or sample examples. For a new RFP, the system retrieves related proposal or response chunks and combines retrieval evidence with structured capacity data, service-domain overlap, and risk flags. In Week 4, the RFP page was updated to pass real or pre-built capacity_df into generate_assignment_context() when available, so recommended directors can be capacity-

aware rather than purely mock-backed. Week 5 QA reconfirmed this handoff and refined the output wording so capacity-aware recommendations are still presented as decision-support guidance rather than final staffing decisions. If only synthetic capacity data is available, the dashboard labels this clearly and treats the output as fallback prototype guidance.

After CGI provided the remaining Azure configuration values, Azure embedding access was validated with a real smoke test, and the Azure embedding plus Chroma retrieval path was integrated into the active RFP backend. In Week 5, retrieval and configuration validation further confirmed that Azure configuration is readable locally, Azure embedding calls produce vectors, Azure/Chroma retrieval works through backend smoke tests and local proposal-corpus retrieval checks, and `generate_assignment_context()` can return `retrieval_mode` and `retrieval_status` metadata. The Streamlit RFP page reads this backend metadata and displays the active retrieval mode, allowing stakeholders to distinguish Azure/Chroma retrieval from local fallback retrieval. Unless an explicit browser-based Azure/Chroma app-flow run is documented, these Azure/Chroma results should be described as backend smoke-test, sample, and full-corpus retrieval validation rather than full production app validation.

Week 6 improved the traceability of retrieved examples by preserving CGI's new RFP alias field from the updated opportunity spreadsheet and using it for alias-based RFP-to-CRM opportunity linkage when available. The proposal or RFP titles can contain an S-number that maps to the cleaned `rfp_alias` field, allowing retrieved chunks to carry linked CRM context such as `opportunity_id`, `opportunity_owner`, `opportunity_manager`, `service_solution`, `status`, `status_reason`, and `opportunity_outcome` when a deterministic alias match exists. This linkage improves reviewability because stakeholders can inspect semantic retrieval evidence together with related CRM metadata.

However, the linkage should still be interpreted conservatively. Retrieved examples remain semantic supporting evidence related to the submitted RFP text, and linked CRM metadata provides contextual information rather than proof that a recommended director personally authored a proposal, worked on the retrieved historical RFP, owned the opportunity in a staffing sense, or should be as-

signed to the new RFP. If no reliable alias match is available, the retrieved example should still be treated only as semantic evidence. Even when linked metadata is available, recommendations remain prototype decision-support outputs that require CGI review and validation before any staffing decision.

2.4 EDA Findings and Design Implications

The EDA directly influenced several early design decisions. First, because `opps2` has the richer CRM schema and preserves the opportunity-level grain more cleanly, it is used as the base table for `opportunity_df`. Second, because `opps1` contains useful service and product fields but also duplicated opportunity IDs, it is treated as a supplemental source rather than a second base table. Third, because the proposal documents are long, the RFP pipeline uses chunking for the historical or sample corpus before embedding or retrieval-style processing. Finally, because the data does not contain labeled capacity categories or gold-standard RFP assignments, the system uses transparent formula-based scoring and retrieval-supported recommendation logic instead of supervised learning.

By Week 5, these design implications became more concrete for the refined final prototype. The capacity dashboard continues to rely on opportunity-level CRM data and formula-based workload estimation, while the RFP Assignment Tool combines Azure/Chroma or fallback retrieval evidence with structured capacity signals when `capacity_df` is available. The team validated Azure embedding access, confirmed Azure/Chroma backend retrieval over both sample and full proposal-corpus checks, preserved local fallback retrieval for robustness, and refined the Streamlit wording so retrieval mode, LLM enrichment status, capacity-data source, and prototype caveats are clearer to stakeholders.

Week 6 added a handoff-oriented implication to the data design. Because CGI may continue testing the prototype with local updated opportunity spreadsheets, the opportunity cleaner now supports configurable spreadsheet paths while preserving the original demo-data defaults. This keeps the preprocessing workflow reproducible for the team while making it easier for CGI to regenerate processed CRM-derived capacity data locally. The Week 6 updated `opps1` file also introduced the `Json S-Num RFP alias` field, which is normalized and preserved as `rfp_alias`. This field provides a

machine-readable linkage signal that can connect historical proposal or RFP records to CRM opportunity rows when a deterministic alias match is available.

The final prototype therefore distinguishes between the preferred Azure/Chroma retrieval path, local fallback retrieval behavior, heuristic scoring, LLM-enriched explanation fields, semantic retrieval evidence, alias-based CRM context, and remaining business-validation limitations. Retrieved supporting examples can help stakeholders inspect semantic similarity, and Week 6 alias-based linkage can surface related CRM opportunity metadata when available. However, this linkage should not be overinterpreted as confirmed evidence of director experience, proposal authorship, staffing history, CRM ownership in a business-valid sense, or final assignment suitability. The system remains a transparent decision-support prototype whose linked metadata and recommendations require CGI review and validation.

3 System Overview

The project has two core deliverables. The first is a Director Capacity Dashboard. This dashboard uses historical CRM opportunity data to estimate each director's current workload, compare the current workload against the director's historical baseline, and display capacity labels such as Available, Near Historical Norm, High Load, Overextended, and No baseline. In the Week 6 handoff-ready prototype, `relative_load` remains the primary business-facing workload metric because it directly compares a director's current workload against that same director's historical norm.

The dashboard also exposes drivers of the score, including open opportunities, late-stage opportunities, weighted pipeline, and inferred delivery commitments. This is important because stakeholders need not only a final capacity label, but also an explanation of why a director appears available, near their normal workload, highly loaded, or overextended. The dashboard keeps `capacity_score` in the backend for compatibility, but the main dashboard interpretation is now based on `relative_load`.

The second deliverable is an RFP Assignment Tool. For a new RFP, the tool estimates effort, retrieves supporting proposal or response examples from the historical or fallback corpus, and recommends directors using re-

trieval evidence, available capacity signals, service-domain overlap, and risk flags. By the final integration stage, the RFP page was updated to pass real or pre-built `capacity_df` into `generate_assignment_context()` when available, so the recommended directors can be capacity-aware rather than purely mock-backed. Week 5 QA reconfirmed this capacity handoff and refined the stakeholder-facing wording so recommended directors are presented as prototype decision-support suggestions rather than final staffing decisions. In Week 6, the RFP workflow gained additional traceability by allowing retrieved examples to carry linked CRM opportunity context when a deterministic `rfp_alias` match is available. The RFP output also supports Azure OpenAI chat enrichment for the RFP summary, effort estimate, and director match reasons. If chat enrichment is unavailable, fails, or returns malformed output, the system falls back to deterministic heuristic output with the same dashboard-facing schema.

The current architecture is modular. The opportunity cleaning pipeline produces a local `opportunity_df` and a cleaned downstream dataset for validation and scoring. In Week 6, this pipeline was updated to support command-line spreadsheet inputs for client data replacement, including configurable local `ops1` and `ops2` paths, sheet name, and processed-output directory, while preserving the original no-argument demo workflow. The cleaner also preserves CGI's new machine-readable RFP alias field as `rfp_alias / has_rfp_alias` when present. The capacity scoring module consumes the cleaned opportunity data to compute director-level workload, `relative_load`, `capacity_score`, and capacity labels. The Streamlit dashboard displays capacity metrics and provides an interface for RFP assignment. The RFP pipeline preprocesses proposal and response text, creates historical or sample corpus chunks for retrieval, supports the preferred Azure/Chroma backend retrieval path when available, preserves local fallback retrieval, propagates linked CRM metadata when alias-based linkage is available, and produces a dashboard-facing `assignment_context` schema for downstream recommendation display.

After CGI provided the remaining Azure configuration values in Week 4, Azure embedding access was validated through a real smoke test, and Azure-generated embeddings were successfully added to and queried from Chroma. In Week

5, retrieval and configuration validation further confirmed that Azure/Chroma retrieval works through backend smoke tests and local proposal-corpus retrieval checks, including `retrieval_mode / retrieval_status` metadata returned through `generate_assignment_context()`. The local fallback retrieval path remains available if Azure embedding, Chroma storage, Chroma querying, or vector-store setup is unavailable. The RFP page reads and displays the active retrieval mode from the backend output so stakeholders can distinguish Azure/Chroma retrieval from local fallback retrieval.

Therefore, the final prototype should be interpreted as a transparent stakeholder-facing decision-support prototype with capacity-aware recommendations when capacity data is available, LLM-enriched output when configured, Azure/Chroma retrieval when the preferred backend path is active, fallback-safe behavior, alias-based CRM context when available, and clearly documented limitations. Week 5 evidence review and usability review clarified that retrieved examples are semantic supporting evidence and, when linked, CRM context; they are not proof of prior director involvement, proposal authorship, delivery responsibility, or final assignment suitability. Week 6 improved traceability by preserving the RFP alias field and propagating linked CRM metadata where a deterministic alias match exists, but this linked context should still not be interpreted as proof of proposal authorship, director experience, staffing ownership, or final assignment suitability. The prototype still requires CGI validation before any staffing or assignment decisions.

4 Weekly Implementation Progress

4.1 Week 1: Data Foundation and Module Skeletons

During Week 1, the team focused on building the first reproducible data foundation and creating skeletons for the main system components. The goal was not to finalize the full capacity model, dashboard, or RFP assignment tool. Instead, we prioritized stable data structures, module boundaries, and interfaces that can support later development.

The work was divided across five roles. Yixiao focused on opportunity data merging, schema comparison, and duplicate / unmatched record summaries. Kian focused on validation of revenue, date, duration, probability, status, and sales-stage fields.

Merge statistic	Value
opps1 rows	8,435
opps2 rows	8,695
opps1 unique IDs	7,679
opps2 unique IDs	8,695
Matched opportunity IDs	7,628
opps1-exclusive IDs	51
opps2-only IDs	1,067
Final opportunity_df rows	8,746
Duplicate IDs in final output	0

Table 2: Week 1 opportunity merge audit summary. The final output preserves opportunity-level grain by avoiding one-to-many expansion from duplicated opps1 records.

Lyken focused on the capacity scoring skeleton and director table prototype. Freya focused on the Streamlit dashboard skeleton and RFP page placeholder. Jai focused on the RFP preprocessing and vector store skeleton.

For the opportunity data merge, we implemented a reproducible pipeline that loads the two anonymized CRM Excel exports, normalizes column names, compares schemas, uses opps2 as the base opportunity-level table, and supplements selected fields from opps1. The pipeline uses `opportunity_id` as the required join key.

A key Week 1 finding was that opps1 contains duplicate `opportunity_id` rows. These duplicates likely reflect service-level or product-level detail rather than separate opportunities. A direct merge from opps1 into opps2 created one-to-many expansion, which would risk double-counting workload in downstream capacity scoring.

To preserve the intended opportunity-level grain, we collapsed opps1 supplemental fields to one row per `opportunity_id` before merging. Duplicate and detail rows from opps1 are preserved in local audit outputs instead of multiplying the final `opportunity_df`. The final local `opportunity_df` contains 8,746 rows, matching the expected no-expansion count of 8,695 opps2 base rows plus 51 opps1-exclusive opportunity IDs. The final merged dataset contains zero duplicate `opportunity_id` rows.

All generated processed data files are kept local and are not committed to GitHub because they are derived from sensitive CGI data. The reproducible code and tests are committed instead, so other team members with local access to the source files can regenerate the same outputs.

4.2 Week 2: From Skeletons to First Integrated Prototype

In Week 2, the team moved from separate data and module skeletons toward a first integrated prototype. The main development path connected the cleaned opportunity data to validation, owner-level capacity scoring, and Streamlit dashboard display. In parallel, the RFP pipeline progressed from placeholder functions toward sample text preprocessing, chunking, and retrieval-style output.

Yixiao refined the Week 1 merge output into a cleaner downstream dataset for validation, scoring, and dashboard development. This work focused on stabilizing `cleaned_opportunity_df`, confirming the required columns for downstream modules, preserving source and quality flags, and preparing an owner-level base summary using `opportunity_owner`. The goal was not to redesign the Week 1 merge logic, but to make the merged data easier for Kian's validation work, Lyken's scoring module, and Freya's dashboard integration to consume.

Kian extended the validation work from basic field inspection to scoring-oriented field assessment. The validation process reviewed revenue fields, date and duration fields, probability, status, sales stage, `opportunity_owner`, and `opportunity_manager`. The output separates fields that are reliable, fields that are usable with fallback assumptions, and fields that remain risky for formal scoring. This validation layer is important because the capacity model depends on transparent assumptions rather than supervised labels.

Lyken converted the Week 1 `capacity_engine.py` skeleton into a first working capacity scoring workflow. The scoring module computes sales load, inferred delivery load, current owner-level load, historical baseline, relative load, bounded capacity score, and capacity labels. The resulting director-level table provides the first version of `director_capacity_df`, including fields such as `current_load`, `historical_avg_load`, `relative_load`, `capacity_score`, and `capacity_label`.

Freya updated the Streamlit dashboard from a mock-data skeleton toward an integrated capacity dashboard. The Week 2 dashboard reads either the first `director_capacity_df` output or a schema-aligned sample output with the same expected columns. The dashboard displays capacity score by director, capacity labels, current load ver-

sus historical baseline, and an owner-level capacity table. Filters and explanation placeholders were also retained so that the interface can continue to evolve as scoring assumptions are refined.

Jai advanced the RFP pipeline from skeleton files toward an independently runnable prototype. The Week 2 work focused on RFP text preprocessing, chunk generation, chunk metadata structure, and a stable `assignment_context` output schema for dashboard integration. The prototype can produce sample chunks and basic or mock retrieval-style output. Full Azure OpenAI and Chroma integration remain later steps, but the Week 2 interface gives the dashboard a stable structure for retrieved examples, recommended directors, risk flags, and explanatory notes.

Overall, Week 2 established the first end-to-end project workflow: cleaned opportunity data supports validation; validation findings inform capacity scoring; capacity scoring produces a dashboard-ready director table; and the dashboard begins displaying capacity outputs. The RFP pipeline remains partly independent from the capacity workflow, but it now has a clearer preprocessing and output contract for future recommendation integration.

4.3 Week 3: Capacity Dashboard Refinement and RFP Assignment Integration

In Week 3, the team shifted the main development focus from the opportunity data foundation and capacity dashboard prototype toward the RFP Assignment Tool. At the same time, the Director Capacity Dashboard received an important refinement: the dashboard now treats `relative_load` as the primary business-facing workload metric instead of `capacity_score`. This change better communicates overextension because `relative_load` compares a director's current workload against that same director's historical norm, while `capacity_score` is capped once a director reaches or exceeds baseline.

The Week 3 capacity dashboard update also clarified the workload formula and label interpretation. Current load is represented as the sum of sales load and delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and duration weight. Delivery load uses won active deliveries and monthly revenue. Capacity labels are now assigned from `relative_load`: *Available*, *Near Historical Norm*, *High Load*, *Overextended*, and *No baseline*. These thresholds are

Module	Owner	Week 2 update	Main output
Data integration	Yixiao	Refined the Week 1 merge output into a downstream-ready cleaned dataset.	<code>cleaned_opportunity_df</code> , column dictionary, quality flags, and owner-level summary.
Data validation	Kian	Validated fields needed for capacity scoring and dashboard use.	Missingness summary, reliable / risky fields, revenue fallback hierarchy, and date / duration assumptions.
Capacity scoring	Lyken	Implemented the first working owner-level capacity scoring workflow.	<code>director_capacity_df</code> , relative load, capacity score, capacity label, and scoring sanity checks.
Dashboard	Freya	Connected the dashboard to real or schema-aligned scoring output.	Capacity score chart, owner-level table, filters, and explanation placeholders.
RFP pipeline	Jai	Built an independently runnable RFP preprocessing and retrieval-style prototype.	Sample chunks, mock or basic retrieval output, and <code>assignment_context</code> schema.
Consistency pass	Yixiao, Freya	Aligned architecture, capacity scoring, dashboard behavior, RFP retrieval, and output contracts after the latest RFP/capacity merge.	Fixed scoring gates, active-quarter baseline/trend logic, dashboard live-data path, stale RFP retrieval, and related docs/tests.

Table 3: Week 2 progress from separate skeletons toward the first integrated prototype.

project-side prototype choices and should be reviewed by CGI before production use.

The dashboard filtering logic was also refined. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views but do not recompute the historical baseline denominator. This keeps the baseline stable and makes filtered views answer a clearer business question: how the filtered current workload compares with the selected directors' normal workload.

In parallel, the RFP pipeline became the main Week 3 development focus. Based on repository review, the local RFP prototype is partially ready: it can process available proposal text, create chunks, run local fallback embedding or in-memory retrieval, and generate a dashboard-facing `assignment_context`. The active demo path is fallback / heuristic rather than Azure-backed. The remaining integration work focuses on confirming Azure API configuration, completing real Azure OpenAI embedding smoke tests, connecting Chroma retrieval end to end, wiring real capacity data into the RFP engine, refining recommendation logic, and continuing Streamlit RFP page display work.

Yixiao coordinated the Week 3 RFP integration and documentation work. This included reviewing which RFP pipeline components cur-

rently work locally, confirming the intended API and fallback retrieval behavior, aligning the `assignment_context` schema with Jai and Freya, and documenting reproducibility, assumptions, blockers, and risks. A key schema decision is that the backend can keep the current field name effort, while the dashboard displays this object as Estimated Effort. If time allows, `estimated_effort` can be added later as an alias without disrupting existing tests.

Kian completed Sprint 3 implementing a reproducible RFP validation workflow over `proposals_responses.json`. The validation confirmed 19 top-level records and 46 extracted documents (19 proposal documents and 27 response documents), with no malformed top-level records in the current local dataset. The work formalized a chunk metadata contract and validated required fields (`proposal_id`, `chunk_id`, `document_id`, `source_type`, `chunk_index`, `text`, `opportunity_owner`, `opportunity_id`) across 1,216 generated chunks, with zero missing required fields. A key finding is that proposal chunks currently cannot be reliably linked to directors or opportunities because `opportunity_owner` and `opportunity_id` are null in all validated chunks. This limitation is documented explicitly rather than patched with unreliable inferred mapping. Deliverables include `src/rfp_data_validator.py`, `scripts/validate_rfp_data_and_export_samples.py`, `docs/rfp_data_validation.md`, and exported artifacts under `outputs/rfp_validation/` for

Track	Owner	Week 3 focus	Main output
Capacity dashboard refinement	Freya	Implemented and documented the relative_load-first dashboard interpretation and updated capacity-label semantics.	Updated dashboard charts, filter semantics, workload formulas, documentation, and tests.
RFP integration / QA	Yixiao	Coordinated RFP pipeline status, API / fallback behavior, assignment schema, smoke testing, and documentation.	API usage notes, finalized assignment_context schema, smoke test checklist, integration QA notes, and risks.
RFP data validation	Kian	Validated proposals_responses.json, text extraction quality, chunk metadata, and proposal-to-director linkage limitations.	JSON structure summary, chunk metadata dictionary, sample chunks, and RFP data validation notes.
Azure embedding / vector store	Lyken	Advanced the code path from local fallback toward Azure embeddings and Chroma while preserving fallback retrieval; real Azure smoke testing remained blocked by missing API version and deployment configuration.	Azure client helper updates, vector store structure, fallback retrieval behavior, and documented blockers for Azure / Chroma completion.
RFP dashboard integration	Freya	Moved the Streamlit RFP page from placeholder toward displaying assignment_context.	RFP input UI, result display, retrieved examples, recommended directors, risk flags, and display notes.
RFP assignment logic	Jai	Improved generate_assignment_context() toward heuristic recommendation output using retrieval examples, available capacity fields when supplied, and risk flags.	Stable assignment output, heuristic effort logic, recommendation notes, risk flag logic, and test updates.

Table 4: Week 3 progress toward RFP Assignment Tool integration.

Lyken/Jai handoff. Export defaults use redacted text and anonymized IDs for safe sharing.

Lyken focused on the Azure embedding and vector-store path. The Week 3 goal was to move beyond local heuristic embedding toward Azure OpenAI embeddings and Chroma retrieval, while keeping local fallback retrieval available. This work included confirming environment-variable requirements, implementing or updating the Azure client, connecting embeddings to `vector_store.py`, and documenting how the system should behave when Azure is unavailable. At the end of Week 3, real Azure embedding smoke testing was still blocked because the required API version and embedding deployment name had not yet been confirmed, so fallback retrieval remained the stable path for RFP dashboard development.

Freya focused on the RFP page in Streamlit. The Week 3 target was to move the RFP Assignment Tool page beyond a placeholder by displaying `assignment_context` fields such as RFP summary, estimated effort, retrieved examples, recommended directors, capacity signals, match reasons, supporting chunks, and risk flags. The page also needed to handle empty retrieval results or API fail-

ures gracefully and clearly label prototype, heuristic, fallback, or mock outputs.

Jai focused on improving `generate_assignment_context()` and the recommendation logic. The Week 3 output used retrieved examples as supporting evidence and could incorporate capacity fields when director data was supplied. However, the Streamlit RFP page had not yet been updated to pass the loaded capacity data into the engine, so the Week 3 recommendations were still best described as heuristic or mock-backed prototype output rather than fully capacity-aware assignment guidance.

Overall, Week 3 moved the project from a capacity-dashboard-centered prototype toward a broader integrated decision-support tool. The dashboard gained a clearer relative-load-first business interpretation, while the RFP Assignment Tool gained a more explicit path from local fallback retrieval toward Azure embedding, Chroma retrieval, assignment schema stabilization, recommendation logic, and Streamlit display. These Week 3 limitations directly shaped the Week 4 priorities: final integration QA, real capacity-data handoff into the RFP engine, Azure configuration follow-up, LLM

enrichment, retrieval smoke testing, and clearer documentation of fallback and prototype behavior.

4.4 Week 4: Final Prototype Stabilization and Architecture-Complete RFP Flow

In Week 4, after CGI provided the remaining Azure configuration values, the team moved from a polished fallback prototype toward an architecture-complete stakeholder-facing prototype. The goal was still not a production-ready or business-validated assignment system, but rather to complete the intended prototype architecture: real capacity-aware RFP recommendations, Azure OpenAI embedding and Chroma retrieval in the active RFP flow when configured, Azure OpenAI chat enrichment for summary and explanation fields, preserved local fallback behavior, and clear limitations.

Yixiao led final integration QA, scope freeze, and documentation. This work defined the final prototype scope, demo path, claims the team can honestly make, claims the team should not overstate, architecture compliance notes, reproducibility notes, demo checklist, QA checklist, final limitations, and Week 4 report notes. The documentation distinguishes real local processing, heuristic scoring, Azure-backed retrieval, fallback retrieval, mock or sample data, Azure OpenAI chat enrichment, and remaining production-readiness limitations. Yixiao also coordinated final QA evidence from the other team members so the report could accurately reflect the implemented system rather than overclaiming business validation.

Kian led final data validation and retrieval-evidence validation. Kian re-checked the proposal and response data structure, chunk metadata, and proposal-to-director / proposal-to-opportunity linkage limitations. For the architecture-complete Week 4 flow, Kian implemented a reproducible evidence workflow in `src/rfp_evidence_validator.py` and `scripts/generate_rfp_retrieval_evidence.py`, documented the results in `docs/rfp_final_evidence_validation.md`, and exported review artifacts under `outputs/rfp_evidence/`. The evidence run evaluated eight representative query themes (cloud migration, managed services, data analytics dashboard, cybersecurity compliance, application modernization, service desk support, change management training, and short vague requests), compared Azure / Chroma retrieval against local

fallback on three sample themes, and recorded top retrieved chunk IDs, relevance notes, and weak or misleading retrieval cases using redacted previews rather than raw proposal text. Because full-corpus cloud embedding was constrained by service rate limits during the evidence run, the Azure / Chroma path was validated on a representative chunk subset, while local fallback searched the full historical corpus. A key limitation remains that retrieved proposal chunks should not be overinterpreted as proven director-experience evidence when `opportunity_owner` or `opportunity_id` linkage is incomplete in all validated chunks.

Lyken led the Azure / Chroma retrieval backend integration. After the required Azure configuration values became available, Lyken confirmed the Azure environment, validated Azure embedding access, and moved the Azure OpenAI embedding plus Chroma retrieval path from smoke-test status into the active RFP backend. The RFP engine now prefers Azure-generated embeddings and Chroma retrieval when the required configuration is available, while preserving deterministic local fallback retrieval if Azure embedding, Chroma storage, or Chroma querying is unavailable. Lyken also documented retrieval-mode behavior, vector-store rebuild/query steps, backend limitations, and the fallback path that keeps the final demo reproducible.

Freya led final RFP output quality, dashboard polish, and Azure OpenAI chat enrichment. The RFP page was updated to a Week 4 final prototype presentation, including clearer retrieval, capacity-data, and recommendation-status labels; upload/paste workflow labels workflow labeling; a consolidated capacity/retrieval notice; and severity-specific risk flag styling. Freya connected the RFP page to real or pre-built capacity data by passing `capacity_df` into `generate_assignment_context()` via `director_df=capacity_df`. This allows recommended directors to use real capacity signals when available rather than relying only on mock director data. When Azure chat is configured and RFP text is non-empty, `generate_assignment_context()` can use the chat deployment to enrich the RFP summary, effort estimate, and per-director match reasons. If chat enrichment is unavailable, raises an error, returns malformed JSON, returns partial JSON, or returns invalid nested fields, the system falls back to deterministic heuristic output while preserving the same dashboard-facing `assignment_context` schema. The

output also preserves backward compatibility by keeping `effort["rationale"]` aligned with `effort["reason"]`.

Jai led RFP page implementation support and stability testing. Jai verified that the Streamlit RFP page can display the agreed `assignment_context` fields, including RFP summary, Estimated Effort, retrieved examples, similar RFPs, recommended directors, risk flags, and notes. Jai also checked that recommended-director cards can display capacity label, capacity score, relative load, assignment score, match reason, capacity explanation, experience-match explanation, and supporting chunks. For the architecture-complete flow, Jai verified that the page labels the active retrieval mode, remains stable when Azure / Chroma falls back to local retrieval, handles empty input and missing fields, and does not crash when LLM enrichment or capacity data is unavailable.

A key Week 4 integration fix connected the Director Capacity Dashboard output to the RFP Assignment Tool. The Streamlit app was already loading `capacity_df`, but the RFP page had previously called `generate_assignment_context(rfp_text)` without passing the loaded capacity data into the assignment engine. The call was updated to `generate_assignment_context(rfp_text, director_df=capacity_df)`, allowing recommended directors to use real or pre-built capacity signals when available. The RFP results area also tracks and displays the capacity data source used for the recommendation, distinguishing live CRM capacity data, pre-built `director_capacity_df.csv`, and synthetic mock or fallback capacity data.

After the Azure embedding and Chroma retrieval path was integrated, the active RFP flow became architecture-complete at prototype level. The preferred path now uses Azure OpenAI embeddings and Chroma retrieval when configured, then combines retrieved examples with real capacity data and optional Azure chat enrichment to produce the dashboard-facing `assignment_context`. The local fallback retrieval path remains available for reproducibility and failure handling. The Streamlit RFP page labels retrieval status so stakeholders can distinguish Azure-backed retrieval from fallback retrieval.

Local QA and manual demo validation were completed after the Week 4 changes. The Director Capacity Dashboard launched successfully,

loaded live CRM capacity data, and displayed director-level capacity outputs including capacity labels, KPI cards, a relative-load chart, and a label-mix chart. The RFP Assignment Tool accepted pasted sample RFP text, retrieved relevant examples through the active retrieval path when configured, generated an RFP summary, displayed estimated effort, recommended directors using live or pre-built capacity data when available, displayed risk flags, and labelled the retrieval and capacity-data status. The final full-suite test result should be reported from the last Week 4 QA run.

Overall, Week 4 moved the project from a Week 3 fallback / heuristic RFP prototype to an architecture-complete stakeholder-facing capstone prototype. The capacity dashboard remained stable around the relative-load-first interpretation. The RFP Assignment Tool became capacity-aware when real or pre-built capacity data is available, integrated Azure OpenAI embedding and Chroma retrieval into the active RFP flow when configured, gained Azure OpenAI chat enrichment for summary, effort, and match-reason fields, preserved local fallback behavior for robustness, and added stronger empty-state, error-state, retrieval-quality, and response-quality checks.

4.5 Week 5: User-Style Testing, Evidence Review, and Final Refinement

In Week 5, the team shifted from architecture-complete feature development toward user-style testing, issue fixing, CGI feedback tracking, and final refinement. The goal was not to add major new functionality, but to validate whether the stakeholder-facing prototype could be demonstrated clearly, whether the RFP outputs were supported by appropriate evidence, and whether the app wording avoided overclaiming production readiness, director experience, or assignment authority.

Yixiao led final app QA and scope tracking. This work consolidated manual QA results, automated test status, issue / fix / decision tracking, teammate follow-up, CGI feedback placeholders, and final scope / caveat alignment. The Week 5 automated baseline was recorded as 149 passed, 2 warnings. Manual QA confirmed that the Director Capacity Dashboard loaded successfully, displayed computed CRM-derived capacity data, rendered key charts, and used appropriate prototype framing. The RFP Assignment Tool was browser-tested with three pasted sample inputs

Track	Owner	Week 4 focus	Main output
RFP contract consistency pass	Yixiao, Freya	Tightened the Week 3 RFP assignment contract before final integration.	Stable assignment_context; effort.reason primary and rationale alias; safer fallback chunks; non-finite numeric cleanup; regression tests; 108 passed.
Final integration QA	Yixiao	Froze final prototype scope, demo path, safe claims, limitations, and reproducibility notes.	QA checklist, demo checklist, architecture-compliance notes, final caveats, and report-ready scope language.
RFP data validation	Kian	Validated proposal structure, chunk metadata, retrieval examples, and linkage limits.	Final validation summary, representative retrieval examples, chunk metadata notes, and proposal-to-director / opportunity linkage caveats.
Azure / Chroma retrieval	Lyken	Integrated Azure OpenAI embeddings and Chroma retrieval into the active RFP flow, with fallback preserved.	Azure environment check, embedding validation, Chroma retrieval path, retrieval-mode notes, and fallback behavior.
RFP output quality and LLM enrichment	Freya	Polished RFP UI, wired real capacity data, and added Azure chat enrichment.	Week 4 labels, upload/paste labels, severity risk flags, capacity_df handoff, LLM-enriched summary/effort/match reasons, and fallback-safe tests/docs.
RFP page stability	Jai	Verified page display and stability across normal, fallback, missing-field, empty, and error states.	Assignment-context display checks, sample RFP tests, retrieval-mode display, and stability notes.
Final prototype validation	Freya	Finalizes the architecture-complete prototype.	Improved upload, retrieval, service-fit, LLM enrichment, and fallback behavior; documented in final_prototype_validation.md.

Table 5: Week 4 progress toward final prototype stabilization and architecture-complete RFP assignment flow.

covering cloud migration / managed services, cybersecurity / compliance, and a vague short technology request. This work also reconfirmed that the RFP path passes `director_df=capacity_df` into `generate_assignment_context()` when capacity data is available, while documenting remaining caveats around upload input, empty input, large input, CGI feedback, and full browser-based Azure/Chroma app-flow verification. Yixiao also implemented a low-risk performance and stability refinement for interactive RFP retrieval: the app no longer rebuilds the Chroma collection by default on every Analyze RFP request, attempts to reuse an existing Chroma store or collection when available, supports explicit rebuild behavior through a configuration flag, and falls back to local retrieval if Chroma is unavailable, not installed, missing, or fails during reuse. This keeps the basic Streamlit demo path robust without making chromadb a hard runtime dependency, while preserving Azure/Chroma backend validation when dependencies and configuration are available.

Kian led evidence and output review. The review tested three representative RFP inputs and checked whether retrieved examples supported the displayed outputs. The cloud migration/managed services case produced mixed evidence because some highly ranked chunks were semantically related but came from broader or differently titled proposal material. The cybersecurity/compliance case produced mostly relevant evidence. The vague short input produced weak or broad evidence, reinforcing the need for low-confidence warnings when the submitted RFP text lacks detail. A key Week 5 conclusion is that retrieved chunks are useful as semantic evidence, but they should not be treated as proof of prior director involvement, confirmed director experience, or CRM opportunity ownership because reliable proposal-to-director and proposal-to-opportunity linkage is still unavailable.

Lyken led retrieval and configuration validation. The Week 5 checks confirmed that the local Azure configuration is readable, Azure embedding calls return valid vectors, Azure/Chroma

retrieval works through backend smoke tests and local proposal-corpus retrieval checks, and `generate_assignment_context()` can return `retrieval_mode` and `retrieval_status` metadata. The full-corpus backend validation used 1,216 proposal chunks and returned `retrieval_mode = azure_chroma`. The scripted Azure/Chroma smoke test also passed. The local fallback path remains available and is still clearly distinguished from Azure/Chroma retrieval. Week 5 documentation therefore treats Azure/Chroma as validated at backend smoke-test, sample, and full-corpus levels, while avoiding a stronger claim of full browser-based Streamlit Azure/Chroma app-flow validation unless that exact UI path is explicitly run and documented.

Freya led stakeholder-facing usability and wording refinement. The capacity dashboard screenshots were reviewed as understandable and reasonable for stakeholder demonstration. For the RFP Assignment Tool, Freya refined wording and display choices to reduce overclaiming and improve interpretability. The results section label was changed from Similar Historical RFPs to Retrieved Supporting Examples, retrieved-result scores were described as match scores, fallback captions were rewritten in client-facing language, and the corpus caption was generalized from pasted-RFP query chunks to submitted RFP query text with historical or sample corpus chunks. Freya also separated retrieval status from LLM enrichment status so that local retrieval fallback is not confused with Azure OpenAI chat fallback. Recommended-director metrics were rounded, and LLM match-reason guidance was updated so that director rationales are framed as prototype fit based on capacity, service-fit signals, and retrieved semantic evidence rather than proven prior experience.

Jai led assignment logic review and RFP status documentation. Jai tested recommendation behavior with varied RFP inputs, including cloud/data modernization, cybersecurity, vague IT consulting, procurement/deadline-only text, a large multi-domain transformation RFP, empty input, TXT upload, and fallback mode. These tests confirmed that the tool can produce coherent outputs across several realistic and stress-test scenarios, while also documenting remaining caveats. First-time real-corpus Azure/Chroma index creation, explicit rebuilds, or missing local vector-store state can still be slow, so cache management, index persistence, and prebuilt Chroma stores remain recom-

mended follow-up areas. Vague or broad RFP text can still produce confident-looking service-fit signals, so stronger low-detail risk flags may be useful. Jai also checked proposal-to-CRM crosswalk candidates and found no strong automatic CRM match for most proposal/RFP entries, supporting the current caveat that retrieved chunks should be treated as semantic evidence rather than confirmed director-history evidence.

Overall, Week 5 refined the final prototype through a validation and de-risking loop: retrieval/config checks confirmed that the preferred backend path can operate with fallback safety; evidence review clarified which outputs are supported by retrieved examples; assignment wording was softened where needed; stakeholder-facing labels and captions were made clearer; and final QA tracking consolidated test results, open caveats, and safe report language. The resulting system is better positioned for CGI review because it distinguishes capacity-aware recommendation signals, retrieved semantic evidence, LLM-enriched wording, local fallback behavior, and remaining business-validation limits.

4.6 Week 6: Client Data Replacement and Handoff Support

In Week 6, the team shifted from final app QA and refinement toward client handoff support, with a focus on making the prototype easier for CGI to test with locally provided opportunity spreadsheets. The goal was not to change the capacity scoring formula, dashboard metrics, Azure/Chroma retrieval ranking, or RFP assignment logic. Instead, the team focused on client data replacement, RFP alias preservation, RFP-to-CRM opportunity linkage, regression testing, app-loading validation, and handoff documentation. This kept the project within a conservative scope: the system remains a stakeholder-facing decision-support prototype rather than a production staffing system.

Yixiao led the client data replacement pipeline work. The opportunity cleaner was updated so CGI can run preprocessing with local spreadsheet paths through command-line arguments, including configurable `opps1` and `opps2` paths, sheet name, and processed-output directory. The original no-argument workflow was preserved, so the cleaner still defaults to the demo/anonymized filenames when no custom paths are supplied. This design allows CGI to place local spreadsheets under the local data directory, regenerate processed CRM-

Track	Owner	Week 5 focus	Main output
Final app QA and scope tracking	Yixiao	Coordinated user-style QA, issue/fix/decision tracking, teammate follow-up, and final claim-boundary alignment.	Final QA tracker, automated baseline of 149 passed, 2 warnings, manual dashboard and RFP checks, scope/caveat notes, Chroma reuse and fallback-safe retrieval stability fix, and report-ready safe wording.
Evidence and output review	Kian	Reviewed whether retrieved examples support displayed RFP outputs across cloud, cybersecurity, and vague short input cases.	Evidence review notes, effort-support checks, weak-evidence cases, and semantic-evidence boundary language.
Retrieval and config validation	Lyken	Validated Azure configuration, Azure embedding, Azure/Chroma retrieval, retrieval metadata, and fallback behavior.	Backend smoke tests and local proposal-corpus retrieval checks; retrieval_mode/retrieval_status reporting; fallback-safe retrieval behavior.
Dashboard usability and wording	Freya	Reviewed stakeholder-facing dashboard and RFP output clarity, including retrieval, fallback, LLM, and linkage caveat wording.	Retrieved Supporting Examples label, match-score wording, separate LLM enrichment status, rounded director metrics, and safer director-experience caveats.
Assignment logic review	Jai	Tested recommendation behavior, risk flags, fallback mode, upload behavior, and proposal-to-CRM linkage assumptions.	Varied RFP test notes, softened match-reason wording, fallback test result, remaining broad/vague-input caveats, and proposal-to-CRM crosswalk limitation notes.

Table 6: Week 5 progress from architecture-complete prototype development toward user-style testing, evidence review, final wording refinement, and scope/caveat consolidation.

derived outputs under data/processed, and run the Streamlit app without committing raw spreadsheets or generated processed data to GitHub. As part of this work, the updated opps1 file was inspected and the new Json S-Num field was preserved through preprocessing as rfp_alias, with has_rfp_alias marking rows where the alias is present. Local validation confirmed that the 19 non-empty RFP alias values in the updated spreadsheet are preserved in the cleaned output.

Freya led the RFP-to-CRM opportunity linkage work. Building on the preserved rfp_alias field, the RFP pipeline can now map proposal or RFP title S-numbers to cleaned opportunity rows when a deterministic alias match is available. Linked metadata such as rfp_alias, opportunity_id, opportunity_owner, opportunity_manager, service_solution, status, status_reason, and opportunity_outcome can be attached to proposal chunks and propagated through retrieval outputs. This improves traceability by allowing stakeholders to inspect retrieved semantic evidence together with related CRM context. The implementation and documentation still avoid overclaiming: linked CRM metadata is treated as contextual support for review, not as proof of

proposal authorship, director experience, delivery responsibility, or final assignment suitability. Orange-highlighted spreadsheet rows are also treated as visual annotation only and are not used as pipeline logic.

Kian contributed regression and evidence-oriented review. The updated tests cover the default demo workflow, command-line spreadsheet input behavior, configurable sheet-name behavior, missing file or missing sheet cases, RFP alias preservation, and related preprocessing behavior using synthetic fixtures rather than actual CGI data. This helps ensure that the client data replacement workflow does not break the original preprocessing path or leak sensitive client data into version control. Kian’s evidence and output review also supported the conservative interpretation of retrieved examples, especially when evidence is weak, non-won, unclear, or only semantically related.

Lyken validated that the app still works after data replacement. The app-level checks focused on whether processed outputs can be regenerated, whether the Director Capacity Dashboard continues to load processed CRM-derived capacity data, whether key charts and owner-level tables render, and whether the RFP Assignment Tool still loads

and runs after the data replacement workflow. This validation confirmed that the Week 6 preprocessing changes did not require changes to the dashboard metrics or retrieval ranking logic.

Jai focused on handoff troubleshooting and client-guide review. From a CGI user perspective, this work checked whether the new command-line workflow is understandable, whether common failure cases are documented, and whether the client guide explains where local spreadsheets should be placed, how preprocessing should be rerun, why client data should remain local, and what fallback behavior to expect if processed outputs are missing. This documentation work supports the handoff goal of making the prototype easier for CGI to rerun and troubleshoot independently.

The Week 6 full test suite passed with 168 passed, 2 warnings. The remaining warnings come from existing capacity-engine tests and do not indicate failures in the client data replacement workflow. Overall, Week 6 improved the prototype's handoff readiness: CGI can use local opportunity spreadsheets through configurable preprocessing inputs, the new RFP alias field is preserved for linkage review, retrieved examples can carry CRM context when deterministic linkage is available, the app continues to load processed CRM-derived capacity data, and documentation now better explains the safe client data replacement workflow and conservative interpretation of linked evidence.

5 Current Methodology and Prototype Design

The current capacity model is transparent and formula-based. Current workload is represented as the sum of sales load and inferred delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and project duration. Delivery load uses won active deliveries and monthly revenue as a proxy for delivery commitment. This approach keeps the scoring logic interpretable and avoids using an LLM as the source of truth for numeric capacity scores.

In the Week 6 handoff-ready prototype, `relative_load` remains the primary business-facing capacity metric. It is defined as a director's current workload divided by that same director's historical average workload. This interpretation is useful for communicating overextension because it can distinguish a director who is slightly above

normal workload from one who is carrying multiple times their historical norm. `capacity_score` remains in the backend output for compatibility and for downstream RFP assignment scoring, but it is no longer treated as the main dashboard metric because it is capped once `relative_load` reaches or exceeds the baseline.

Capacity labels are assigned from `relative_load`. The current prototype labels directors as *Available*, *Near Historical Norm*, *High Load*, *Overextended*, or *No baseline*. These thresholds are project-side calibration choices based on the observed director-level distribution and the business interpretation that carrying at least twice the historical workload should be considered overextended. CGI should review and adjust these thresholds before production use.

The dashboard filtering logic is designed to keep the historical baseline stable. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views, but do not recompute the historical baseline denominator. This allows filtered views to answer how a selected slice of current workload compares with the selected directors' normal workload.

The Streamlit dashboard delegates capacity calculations to shared functions in `capacity_engine.py`. For live filtered views, the interface rebuilds current workload from filtered opportunity rows while keeping the historical baseline anchored to the selected owner / territory population. This keeps the scoring logic centralized in the engine while allowing the dashboard to support interactive filtering. By the final integration stage, the resulting `capacity_df` is passed into the RFP assignment engine when available, connecting the Director Capacity Dashboard output to capacity-aware director recommendations. In Week 6, this app-facing methodology was not changed; instead, the preprocessing and handoff workflow was improved so CGI can regenerate processed CRM-derived capacity data from local opportunity spreadsheets before launching or testing the dashboard.

For RFP assignment, the current prototype combines proposal text preprocessing, retrieval-oriented evidence, structured capacity signals, `service_solution` fit, heuristic scoring, optional LLM enrichment, and, when avail-

Track	Owner	Week 6 focus	Main output
Client data replacement pipeline	Yixiao	Added command-line spreadsheet inputs for client-provided opportunity files while preserving the original no-argument demo workflow.	Configurable opps1 / opps2 paths, sheet name, and output directory; local processed-output regeneration under data/processed; preserved rfp_alias / has_rfp_alias; 19 non-empty alias rows retained in cleaned output.
RFP-to-CRM linkage	Freya	Used CGI's machine-readable RFP alias field to connect historical proposal/RFP records to CRM opportunity rows when deterministic alias matches are available.	Alias-based linkage from proposal title S-numbers to rfp_alias; linked CRM meta-data including opportunity_id, opportunity_owner, service_solution, status fields, and opportunity outcome; conservative UI/docs wording that treats linkage as context rather than proof.
Regression and evidence review	Kian	Checked that client data replacement does not break existing preprocessing and that evidence wording remains conservative.	Tests for default and CLI workflows, configurable sheet names, missing file / missing sheet behavior, and alias preservation; evidence-review support for weak, unclear, non-won, or semantic-only retrieved examples.
App loading and processed-data QA	Lyken	Validated that the Streamlit app still works after opportunity data is regenerated through the new preprocessing workflow.	Processed-data loading checks; Director Capacity Dashboard smoke test; RFP page post-replacement smoke test; confirmation that dashboard metrics and retrieval ranking logic were not changed.
Handoff troubleshooting and guide review	Jai	Reviewed the workflow from a CGI user perspective and documented common failure cases.	Client-guide and troubleshooting notes covering local spreadsheet placement, preprocessing reruns, missing processed outputs, missing alias columns, privacy / gitignore warnings, and conservative caveats.
Full-suite validation	Team	Confirmed that the updated handoff-oriented workflow remains compatible with the existing codebase.	Latest full test suite result: 168 passed, 2 warnings; no raw CGI spreadsheet or generated processed output committed.

Table 7: Week 6 progress from final prototype refinement toward client data replacement and handoff readiness.

able, alias-based CRM context. The RFP pipeline extracts proposal or response text from `proposals_responses.json` when available, creates chunks for the local proposal corpus, and uses a built-in sample corpus only when the local proposal corpus is unavailable. New RFP input is accepted through pasted text or TXT, DOCX, and text-based PDF upload. The submitted RFP text is used as query text against the chunked historical or sample corpus, and the dashboard-facing `assignment_context` is produced from retrieved supporting examples, capacity data, service-fit signal, risk flags, optional LLM enrichment, and linked CRM metadata when deterministic `rfp_alias` linkage is available.

The intended RFP recommendation logic combines retrieval evidence with structured direc-

tor capacity signals. By the final integration stage, the Streamlit RFP page was updated to call `generate_assignment_context(rfp_text, director_df=capacity_df)` when capacity data is available. This allows recommended directors to use real or pre-built capacity signals, including `capacity_label`, `capacity_score`, and `relative_load`, rather than relying only on mock director data. If only synthetic mock capacity data is available, the dashboard labels this clearly and the recommendation should be interpreted as fallback prototype guidance. Week 5 QA reconfirmed this capacity handoff and clarified that capacity-aware recommendations are still decision-support outputs requiring stakeholder validation rather than final assignment decisions. Week 6 did not change the recommendation ranking logic; instead, it im-

proved traceability by preserving RFP alias information and allowing linked CRM opportunity context to travel with retrieved examples when available.

The RFP output uses a stable `assignment_context` schema for dashboard display. This schema includes an RFP summary, an effort object displayed as Estimated Effort, retrieved supporting examples, recommended directors, risk flags, and notes. Recommended-director cards can display capacity label, capacity score, relative load, assignment score, match reason, capacity explanation, experience match explanation, and supporting chunks. In Week 6, retrieved examples can also carry linked CRM context such as `rfp_alias`, `opportunity_id`, `opportunity_owner`, `service_solution`, status fields, and opportunity outcome when alias-based linkage is available. This design keeps the RFP page stable even when individual backend components use real data, fallback data, heuristic logic, optional LLM enrichment, or linked CRM metadata.

Week 4 also added Azure OpenAI chat enrichment. When Azure chat configuration is available, `generate_assignment_context()` can use the chat deployment to enrich the RFP summary, effort estimate, and per-director match reasons. When chat enrichment is unavailable, fails, or returns malformed JSON, the system falls back to deterministic heuristic output using the same schema. This keeps the dashboard output robust while improving readability when LLM enrichment succeeds. LLM-generated fields are still treated as prototype-quality explanations rather than validated business analysis. Week 5 usability review also separated retrieval mode from LLM enrichment status in the RFP page so stakeholders can distinguish local retrieval fallback from Azure OpenAI chat fallback.

The preferred retrieval architecture uses Azure OpenAI embeddings with Chroma vector-store retrieval. After CGI provided the remaining Azure configuration values, Azure embedding access passed a real smoke test, sample Azure + Chroma retrieval was validated, and the Azure / Chroma retrieval adapter was integrated into the active RFP engine. Week 5 retrieval and configuration validation further confirmed that Azure/Chroma retrieval works through backend smoke tests and local proposal-corpus retrieval checks, including `retrieval_mode /`

`retrieval_status` metadata returned through `generate_assignment_context()`. When Azure configuration is available and the preferred path succeeds, the RFP engine uses Azure-generated embeddings and Chroma retrieval. When Azure embedding, Chroma storage, Chroma querying, or vector-store setup is unavailable, the system falls back to deterministic local retrieval while preserving the same dashboard-facing `assignment_context` schema. The Streamlit RFP page reads the retrieval mode from the backend output, but full browser-based Azure/Chroma app-flow validation should only be claimed when that exact UI path has been explicitly run and documented.

Week 5 evidence review refined the interpretation of retrieved examples, and Week 6 refined this interpretation further after alias-based linkage was added. Retrieved proposal or response chunks can support semantic similarity discussion. When a deterministic `rfp_alias` match is available, they can also carry related CRM opportunity context for stakeholder review. However, retrieved and linked examples do not prove prior director involvement, confirmed director experience, proposal authorship, delivery responsibility, staffing ownership, or final assignment suitability. The RFP page and report wording therefore describe these items as retrieved supporting examples and linked CRM context rather than proven historical director experience or validated assignment evidence.

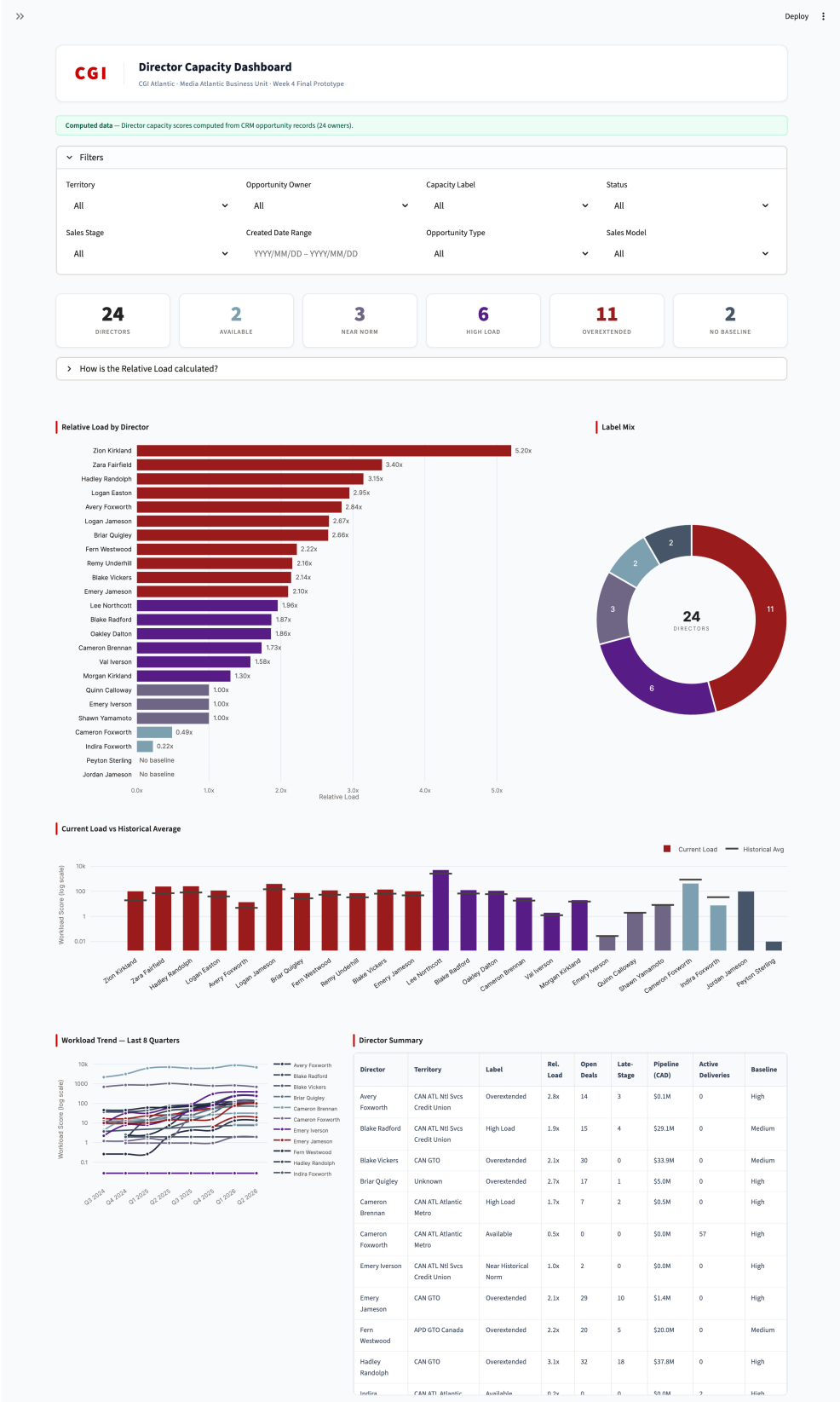


Figure 1: Week 6 Director Capacity Dashboard prototype.

>>

Deploy

CGI

RFP Assignment Tool

CGI Atlantic - Media Atlantic Business Unit - Week 4 Final Prototype

Computed data — Director capacity scores computed from CRM opportunity records (24 owners).

Week 4 Final Prototype — RFP Assignment Tool

Retrieval: active mode is shown after analysis (Azure/Chroma when available, local fallback otherwise). Capacity data: real director capacity scores when available. Recommendations: prototype guidance only — not final CGI assignment decisions.

RFP Input

Upload RFP document

20-09aa-final.pdf

+

Loaded text from 20-09aa-rfp-for-case-management-software-final.pdf.

Upload a supported document or paste RFP text directly below.

schedule:

Activity Date

Pre-proposal Conference Call September 15, 2020 at 11:00am CST

Deadline to submit Questions via email to purchasing@duluthmn.gov

September 21, 2020

Answers to questions will be posted to the City website no later

Territory

Service Domain

Any

Any

Territory and service domain are UI context only for now; backend scoring does not use them yet.

Analyze RFP

Retrieval mode — Azure/Chroma retrieval active.

Prototype output — capacity-aware recommendations using Computed from CRM opportunity records. Retrieval: Azure/Chroma retrieval active.

Azure/Chroma retrieval is active for this result.

Retrieval corpus: local proposals_responses.json corpus, 1216 corpus chunks, 11 submitted RFP query text; historical/sample corpus chunks.

Some retrieved examples are linked to CRM opportunities through CGI's RFP alias. Non-won linked opportunities are historical context, not proof of successful prior delivery. Linked opportunity owner reflects CRM ownership/context, not proof of personal delivery experience.

LLM enrichment: Azure OpenAI chat was used for the summary, effort estimate, and director match wording.

RFP Summary

The City of Duluth is seeking proposals for a case management software solution for its City Attorney's Office, including software, hardware, implementation, training, and ongoing support to replace the current system and improve operational efficiency.

Estimated Effort

High

Estimated duration: 12-16 weeks

The project involves multiple components such as software acquisition, hardware integration, implementation, training, and long-term support, requiring significant coordination and resources.

Retrieved Supporting Examples

1. 4_courts_case_management_system — match score 0.61

Proposal: 4_courts_case_management_system

Chunk ID: 4_courts_case_management_system_proposal_chunk_881

Match score: 0.61

CRM opportunity: 18621656

Opportunity owner: Hadley Randolph

Outcome: open

= 100%. A Proponent who bids 150.00 receives 80% of the possible points for that category (126/150 = 88%), and a Proponent who bids 240.00 receives 50% of the possible points for that category (120/240 = 50%). Lowest rate ----- x Total available points = Score for second-lowest rate Second-lowest rate...

2. 3_data_sharing_platform_shared_analytics — match score 0.59

3. 4_courts_case_management_system — match score 0.59

Recommended Directors

1. Indra Foxworth

Capacity: Available

Capacity score: 0.78

Relative load: 0.22x

Assignment score: 0.68

Prototype fit is based on available capacity, service-fit signals, and retrieved semantic evidence; it does not prove prior director experience. LLM note: Indra's expertise in managing complex projects and her ability to oversee multi-phase implementations make her a strong candidate for this RFP.

Capacity explanation

Indra Foxworth is labelled Available with capacity score 0.78 and relative load 0.22. This is a heuristic input to the assignment score, not a final staffing decision.

Experience match explanation

Experience fit is estimated from retrieved historical/sample RFP chunks and simple service-keyword overlap (0.40). Retrieved examples include CRM opportunity linkage, but not to this director in the current top retrieved set.

Supporting chunks: 4_courts_case_management_system_proposal_chunk_0031, 3_data_sharing_platform_shared_analytics_proposal_chunk_0010

2. Cameron Foxworth

3. Jordan Jameson

Risk Flags

Low: Some retrieved examples are linked to CRM opportunities, but not all linked opportunities are won. Treat non-won links as historical context, not proof of successful prior delivery.

Medium: Some directors have no historical baseline, so capacity score and relative load use heuristic defaults in RFP ranking. Review No baseline candidates before assignment.

High: At least one director in the broader capacity pool is overextended based on capacity signals. Review workload before assigning outside the recommended shortlist.

Notes

Prototype output enriched with Azure OpenAI chat analysis. Retrieval used Azure/Chroma. Retrieved evidence searched the local proposals_responses.json corpus.

Figure 2: Week 6 RFP Assignment Tool prototype.

6 Project Management and Next Steps

In Week 6, the project focused on client hand-off support, data replacement readiness, and RFP-to-CRM opportunity linkage. The team assessed whether CGI could safely replace local opportunity spreadsheets, regenerate processed CRM-derived outputs, preserve the new RFP alias field and derive RFP-to-CRM linkage from it, inspect linked CRM context for retrieved examples, and continue testing the prototype without committing client data or generated outputs to the repository.

Yixiao updated the opportunity preprocessing workflow so CGI can provide local spreadsheet paths through command-line arguments while preserving the original no-argument demo workflow. The updated cleaner supports configurable opps1 / opps2 paths, sheet name, and processed-output directory, preserves the new `Json S-Num` field as `rfp_alias / has_rfp_alias`, and keeps raw CGI spreadsheets and regenerated processed outputs local.

Kian contributed Week 6 regression and evidence-oriented checks, covering default and CLI preprocessing paths, configurable sheet behavior, missing file / missing sheet cases, and RFP alias preservation using synthetic fixtures rather than actual CGI data. This reinforced the evidence boundary that retrieved examples are semantic evidence and, when linked, CRM context; they are not proof of prior director involvement, proposal authorship, delivery responsibility, or final assignment suitability.

Lyken led app-loading validation after data replacement to confirm that the app still works after opportunity data is regenerated through the new preprocessing workflow. These checks covered processed-data loading, Director Capacity Dashboard rendering, RFP page loading after data replacement, and failure-state behavior when processed data is missing or regeneration fails.

Freya led RFP-to-CRM opportunity linkage work. She built on the preserved `rfp_alias` field to support deterministic RFP-to-CRM opportunity linkage when proposal or RFP title S-numbers match cleaned opportunity aliases. Linked metadata can now be propagated with retrieved examples, including `opportunity_id`, `opportunity_owner`, `service_solution`, status fields, and opportunity outcome. Freya also reviewed documentation and UI wording so this linkage is presented as CRM context for review, not as

proof of director experience, proposal authorship, or staffing ownership.

Jai reviewed the new client data replacement workflow from a CGI user perspective, focusing on command clarity, common failure cases, local data placement, privacy / gitignore warnings, rerunning preprocessing before Streamlit launch, and fallback behavior when processed outputs are unavailable.

The latest Week 6 full-suite validation result was 168 passed, 2 warnings. The remaining warnings come from existing capacity-engine tests and do not indicate failures in the client data replacement workflow. The team also continued to keep actual CGI spreadsheets, local credentials, and generated processed outputs out of Git. This supports the handoff design: CGI can test with local data, but the repository remains focused on reproducible code, documentation, and tests rather than committed client data.

The next stage of the project should focus on CGI local testing, stakeholder review, handoff validation, and conservative refinement rather than major new feature expansion. The capacity dashboard thresholds and label names should be reviewed with CGI to confirm whether they match business language. The RFP Assignment Tool should be evaluated with representative stakeholder examples, with particular attention to whether retrieved supporting examples are relevant, whether linked CRM context is understandable, whether risk flags are useful, whether vague-input warnings are strong enough, and whether recommendation wording remains appropriately caveated. Future work should further production-harden the Azure + Chroma retrieval path, including more robust index persistence, cache management, and deployment-time dependency handling; expand retrieval validation across more representative RFPs; validate the new alias-based RFP-to-CRM linkage with CGI; improve document upload parsing for supported file types; calibrate recommendation weights with CGI feedback; and continue refining client-facing hand-off documentation.

7 Current Assumptions and Risks

Several assumptions and risks remain at the end of Week 6. First, `opportunity_id` is still treated as the required join key for opportunity-level merging. Second, `opps2` remains the primary base CRM opportunity table, while `opps1` is used only for selected supplemental fields after be-

ing collapsed to one row per opportunity_id. Third, opportunity_owner is treated as the main director-level entity for MVP capacity scoring, while opportunity_manager is retained as optional metadata. Week 6 did not change these core modeling assumptions; it made the local opportunity preprocessing workflow more configurable for CGI handoff.

The capacity dashboard still depends on inferred workload rather than measured hours, calendar commitments, or manually confirmed director capacity. Current load is estimated from sales pipeline and inferred delivery signals, so the resulting workload estimates should be treated as directional decision-support indicators rather than exact measurements of utilization, workload hours, or guaranteed availability.

The Week 6 handoff-ready dashboard continues to use relative_load as the primary business-facing workload metric. This improves interpretability, but the current label thresholds remain temporary project-side calibration choices. Labels such as Available, Near Historical Norm, High Load, Overextended, and No baseline should be reviewed by CGI before production use. In particular, the threshold for Overextended reflects the current team interpretation that at least twice the historical workload should be treated as overextension.

The stable-baseline filtering decision also remains an assumption. Territory and opportunity-owner filters define the selected director population for both current workload and baseline, while row-level filters narrow current workload without recomputing the historical baseline. This makes the dashboard easier to interpret, but CGI may later decide that some additional filters should also redefine the baseline population.

The Week 6 client data replacement workflow introduces an additional operational assumption. CGI-provided spreadsheets are expected to remain local under project data paths such as data/csv_files/, and generated processed outputs are expected to remain local under data/processed/. The updated opportunity cleaner supports command-line paths for local opps1 and opps2 spreadsheets, a configurable sheet name, and a configurable output directory, while preserving the no-argument demo workflow. However, this remains a local handoff workflow rather than a production data-ingestion service. Raw CGI spreadsheets, local credentials, and generated processed outputs should not be committed

to GitHub.

The RFP Assignment Tool is capacity-aware when the Streamlit page passes capacity_df into generate_assignment_context(). Week 5 QA reconfirmed this app-facing capacity handoff, and Week 6 app-level checks confirmed that processed CRM-derived capacity data can still be loaded after data replacement. However, the resulting recommendations are still prototype decision-support guidance rather than final CGI staffing decisions. The assignment score combines available capacity signals, retrieval evidence, keyword or service-domain overlap, and heuristic logic. These weights have not yet been calibrated against stakeholder-approved outcomes or historical assignment labels.

The project still lacks ground-truth labels for director capacity, RFP effort, and correct director assignment. Evaluation therefore relies on stakeholder review, threshold sensitivity checks, sanity checks, smoke tests, manual demo validation, evidence review, regression tests, app-loading checks, and selected case studies rather than standard supervised-learning metrics. The latest Week 6 full-suite validation result was 168 passed, 2 warnings, but passing tests should be interpreted as implementation and regression confidence, not as business validation of the recommendations.

The RFP retrieval evidence has important limitations. Week 6 improved traceability by preserving the new Json S-Num field from the updated opps1 spreadsheet as rfp_alias / has_rfp_alias and supporting deterministic RFP-to-CRM opportunity linkage when proposal or RFP title S-numbers match cleaned opportunity aliases. When such linkage is available, retrieved examples can carry linked CRM context such as opportunity_id, opportunity_owner, service_solution, status fields, and opportunity outcome. However, this should not be interpreted as proof of proposal authorship, prior director involvement, delivery responsibility, staffing ownership, or final assignment suitability. If no deterministic alias match is available, retrieved examples remain semantic evidence only.

Retrieval quality also depends on the specificity of the submitted RFP text. Week 5 evidence review found that cybersecurity / compliance inputs produced mostly relevant retrieved examples, while cloud migration / managed services produced mixed evidence and vague short inputs produced broad or weak evidence. Short or underspecified inputs can still generate confident-looking sum-

maries, match scores, or service-fit signals. Week 6 linked CRM context can improve traceability when alias matches exist, but it does not solve vague-input or weak-evidence issues. The RFP page should therefore continue to show low-confidence warnings for vague input and should encourage stakeholders to provide enough scope detail before relying on recommendations.

Azure embedding access and Azure + Chroma retrieval were validated after CGI provided the remaining Azure configuration values, and Week 5 retrieval/configuration checks confirmed that the backend path works through backend smoke tests and local proposal-corpus retrieval checks. The system can return `retrieval_mode` / `retrieval_status` metadata through `generate_assignment_context()`. However, this integration remains prototype-level rather than production-hardened. The report should avoid claiming a fully browser-validated Azure/Chroma Streamlit app flow unless that exact UI path has been explicitly run and documented. The system preserves local fallback retrieval when Azure embedding, Chroma storage, Chroma querying, or vector-store setup is unavailable.

Real-corpus Azure/Chroma retrieval still has a performance and dependency-management caveat, but Week 5 added a low-risk stability fix for the interactive path. The app no longer rebuilds the Chroma collection by default on every RFP analysis request. Instead, the retrieval code attempts to reuse an existing Chroma store or collection when available, supports explicit rebuild behavior through a configuration flag, and falls back to deterministic local retrieval when Chroma is unavailable, not installed, missing, or fails during reuse. This makes the basic Streamlit demo path more robust and avoids making chromadb a hard runtime dependency. First-time index creation, explicit rebuilds, or production-scale index management may still require further optimization.

Azure OpenAI chat enrichment introduces an additional interpretation risk. When configured, the LLM can enrich the RFP summary, effort estimate, and director match reasons. These fields improve readability, but they are not validated business analysis. If chat enrichment fails or returns malformed output, the system falls back to deterministic heuristic output. Week 5 usability review separated retrieval mode from LLM enrichment status so stakeholders do not confuse local retrieval fallback with Azure OpenAI chat fallback. The

dashboard and report should continue to label LLM-enriched text as prototype explanation rather than authoritative staffing guidance.

The RFP page still has interface limitations. Week 5 testing confirmed that pasted input and TXT upload can support stakeholder-style testing, and the RFP upload/paste workflow is understandable from a user perspective. However, production-grade document ingestion remains future work. DOCX and text-based PDF parsing should continue to be tested carefully, scanned PDFs may require manual paste or OCR outside the app, and large or malformed inputs require additional validation. Upload support should therefore be described conservatively as prototype-level rather than production-grade document processing.

Finally, the system may combine real data, client-provided local spreadsheets, processed CRM-derived data, heuristic logic, fallback retrieval, sample data, mock capacity data, Azure smoke-test components, Azure/Chroma backend retrieval, linked CRM metadata, and LLM-enriched text during development and demonstration. These components must remain clearly labeled in the dashboard and documentation so that stakeholders understand which parts are implemented, which parts are fallback or heuristic, which parts are linked context rather than proof, which parts are validated only at backend or sample level, and which parts still require future integration or business review.

8 Conclusion

This Week 6 version extends the report from a Week 5 refined stakeholder-facing prototype report into a handoff-ready decision-support prototype update. Week 1 established the reproducible opportunity merge, preserved opportunity-level grain, and created the main module skeletons. Week 2 refined the cleaned opportunity data, added validation-oriented field assessment, produced an initial director-level capacity scoring output, connected the dashboard to capacity results, and advanced the RFP pipeline toward preprocessing, chunking, and retrieval-style output. Week 3 refined the Director Capacity Dashboard around a `relative_load`-first interpretation and shifted the main development focus toward RFP assignment, including fallback retrieval, assignment schema stabilization, heuristic recommendation logic, and Streamlit RFP page integration. Week 4 stabilized the prototype into an

architecture-complete stakeholder-facing version with capacity-aware RFP recommendations, Azure OpenAI chat enrichment, Azure OpenAI embedding and Chroma retrieval integration, preserved local fallback behavior, upload/paste input support, and clearer prototype-status labeling. Week 5 refined this prototype through final app QA, user-style testing, evidence review, assignment caveat refinement, stakeholder-facing wording polish, retrieval/configuration validation, and final scope tracking.

In Week 6, the team shifted from final prototype refinement toward client data replacement and handoff support. The opportunity preprocessing workflow was updated so CGI can provide local opps1 and opps2 spreadsheet paths through command-line arguments while preserving the original no-argument demo workflow. The updated cleaner also preserves CGI's new machine-readable Json S-Num field as rfp_alias / has_rfp_alias, allowing the RFP pipeline to surface linked CRM opportunity context when deterministic alias-based matches are available. Regression tests and app-loading checks confirmed that the handoff-oriented changes remain compatible with the existing workflow, with the latest full-suite validation result recorded as 168 passed, 2 warnings. Documentation and client-facing wording were also refined so the data replacement process, local-only privacy expectations, troubleshooting behavior, and conservative interpretation of linked CRM metadata are clearer for CGI review.

Overall, the project now demonstrates a transparent handoff-ready decision-support workflow: a stakeholder can view director capacity results, inspect workload drivers, submit or paste RFP text, receive capacity-aware prototype recommendations when capacity data is available, review retrieved supporting examples, inspect linked CRM context when reliable alias matches exist, and see risk flags and caveats. The system remains an applied stakeholder-facing prototype rather than a production-ready or business-validated assignment system. Retrieved and linked examples should be interpreted as semantic evidence and CRM context for review, not as proof of proposal authorship, prior director involvement, delivery responsibility, staffing ownership, or final assignment suitability. Future work should focus on CGI review of capacity thresholds and label language; calibration of recommendation weights; larger-scale validation of retrieved examples; CGI validation and refine-

ment of the alias-based RFP-to-CRM linkage; further production hardening of the Azure + Chroma retrieval path, including more robust index persistence, cache management, and deployment-time dependency handling; continued document upload testing; stronger low-detail input warnings; and continued refinement of client-facing handoff documentation.